MOBILE APP DEVELOPMENT 700

RICHFIELD
richfield.ac.za

Name and surname: Tashreef Muhammed Shaik

Student ITS No: 402413638

Qualification: BSC IT Year of Study: 3rd year Semester:2

Assignment due date:24/10/26 Date submitted: 24/10/26

QUESTION	EXAMINER MARKS	MODERATOR MARKS	REMARKS

ASSIGNMENT INSTRUCTIONS

Please tick each box to confirm completion.(10 Marks)

Use Times New Roman font, size 12, with 1.5 line spacing throughout the document.
Apply Harvard Referencing Style for all citations and references.

For essay-style assignments, please include the following sections:

Table of Contents
Introduction
Main Body (with relevant subheadings)
Conclusion
References

Submit the assignment in PDF format on Moodle.
Use the specified cover page provided.
Include a signed declaration of originality.

DECLARATION OF ORIGINALITY:

I hereby declare that this assignment is my own work and has not been copied from any other source except where due acknowledgment is made. I affirm that all sources used have been properly cited and that this submission complies with the institution's policies on academic integrity and plagiarism.

Student Signature: Tashreef M Shaik Date: 28/04/25

Introduction

Problem Statement

Food waste is a major problem in households everywhere, largely because it's so easy to lose track of what's hiding in the back of the pantry. When people forget their ingredients or aren't sure how to combine them, they end up throwing away good food and buying duplicates at the grocery store.

Target Audience

The Smart Pantry Manager app is tailored for home cooks and anyone who wants to:

- Keep a clean, organized inventory of their pantry
- Cut down on everyday food waste
- Save money by making the most of what's already on hand
- Find fresh recipe ideas using current ingredients

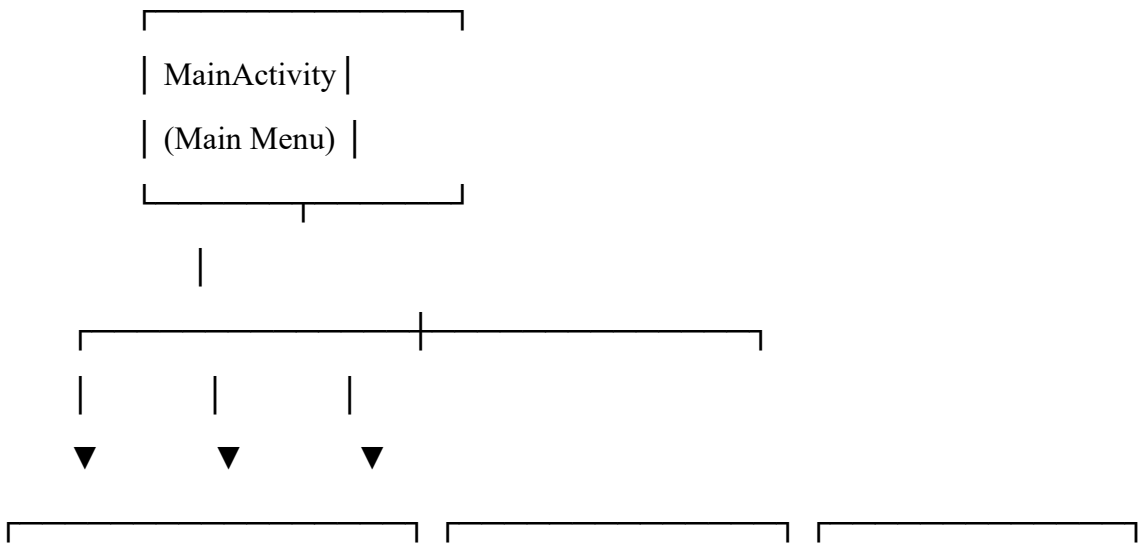
Solution

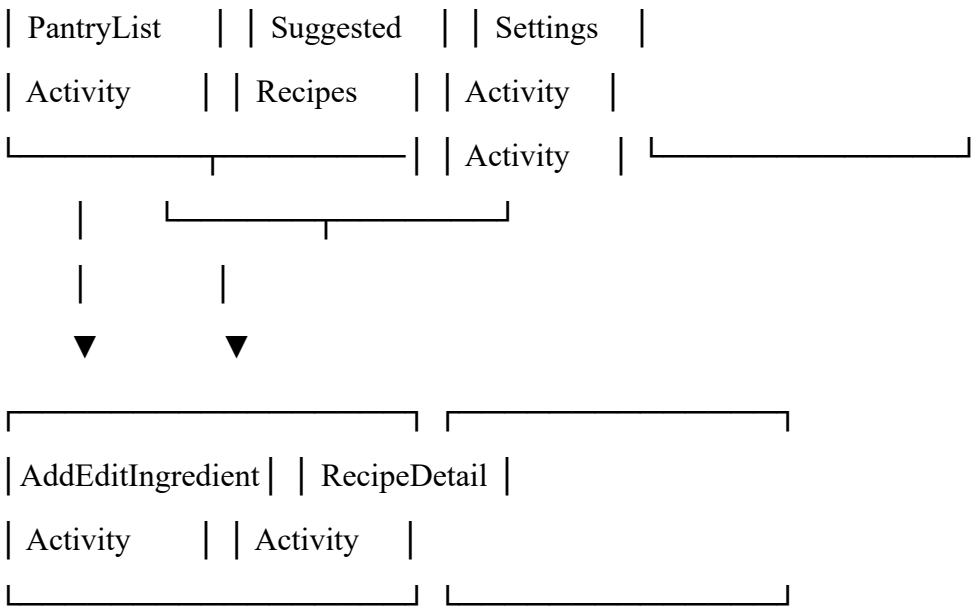
This app tackles the problem head-on by:

- Letting users log pantry items along with their quantities, units, and expiration dates
- Providing a curated built-in collection of 20 recipes
- Running a smart matching algorithm to suggest dishes you can actually make right now
- Inspiring creative ways to use up leftover items before they go bad

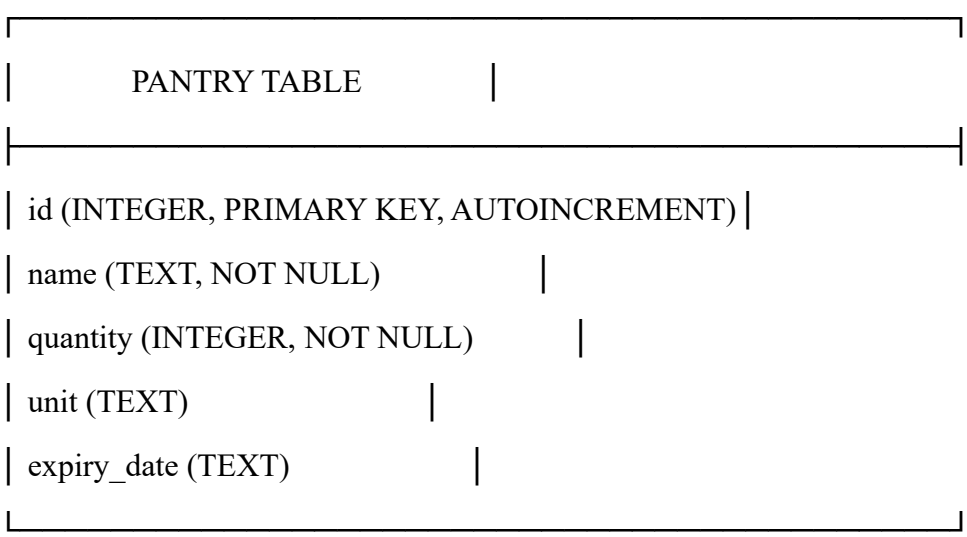
System Design

Screen Flow Diagram





Database ER Diagram



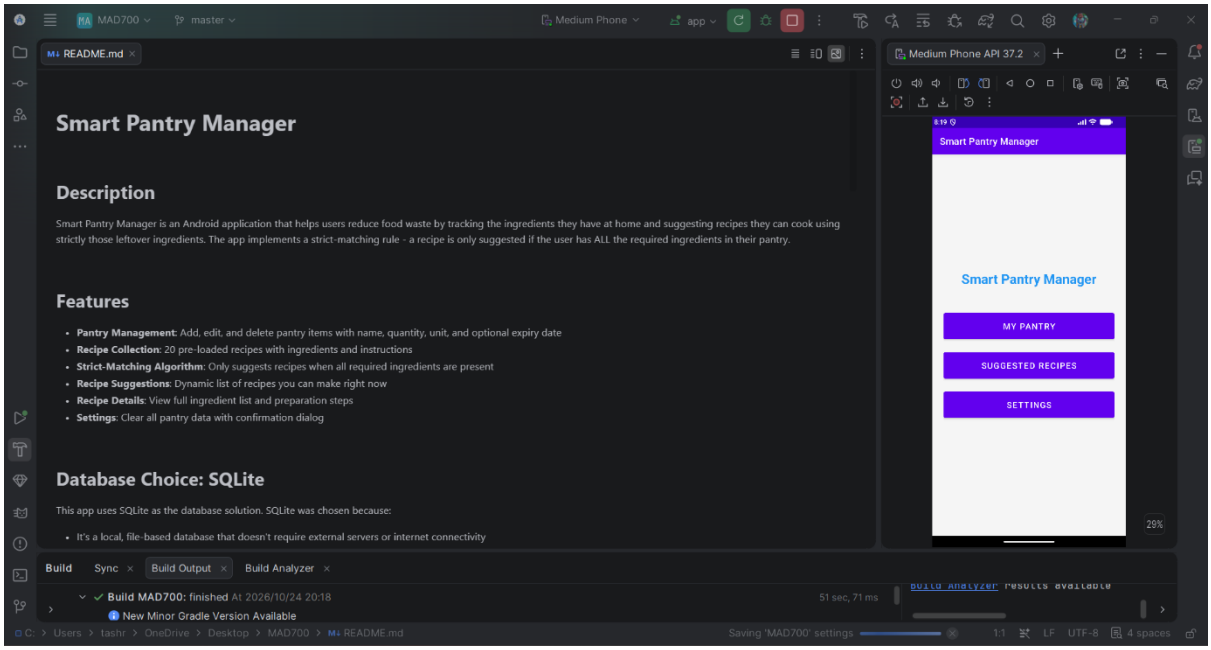
Database Schema:

- **Table Name**: pantry
- **Primary Key**: id (auto-incrementing integer)
- **Fields**:
    - name: Ingredient name (required)
    - quantity: Amount of ingredient (required, integer)
    - unit: Measurement unit (optional, e.g., kg, pieces)
    - expiry_date: Expiry date in YYYY-MM-DD format (optional)

Note: Recipes are stored as Java objects inside the RecipeMatcher class rather than the database, since they act as static reference data that doesn't need to change dynamically.

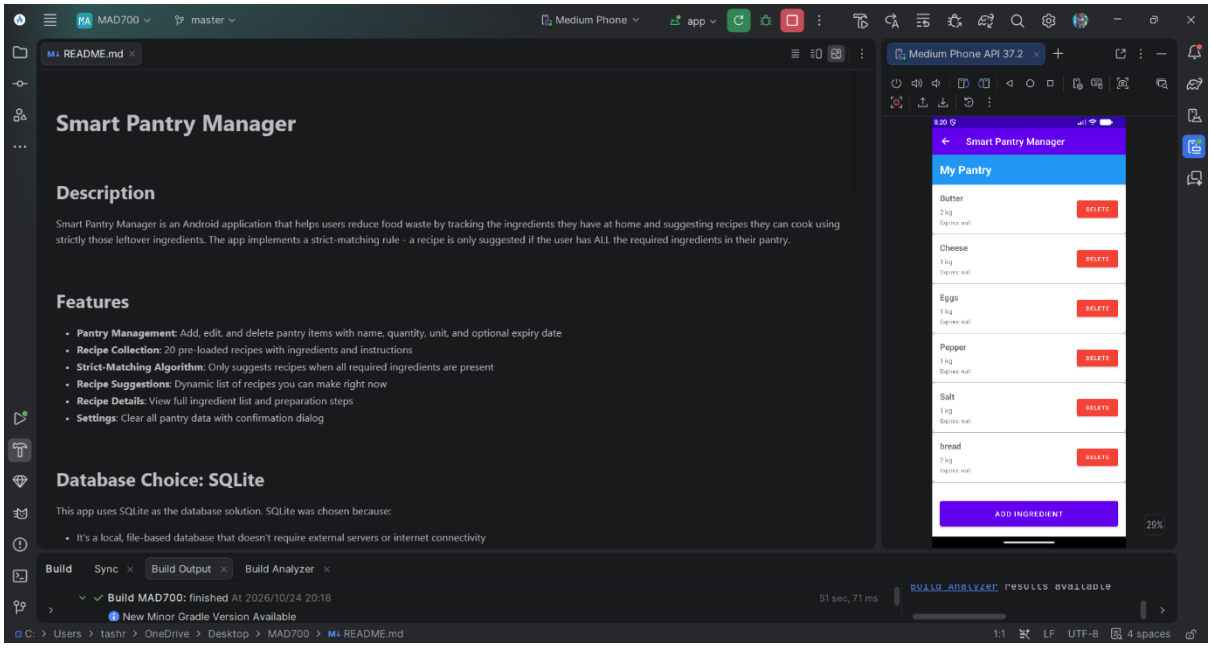
Screenshots

Main Screen



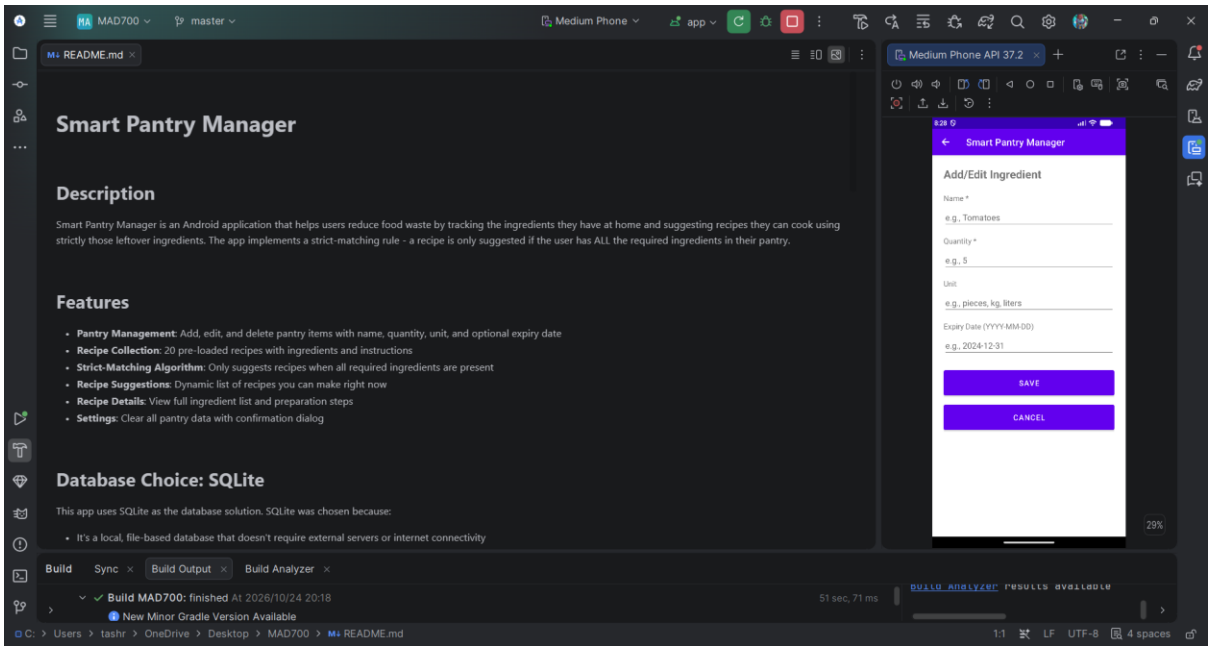
Caption: The main hub screen, offering quick navigation to your pantry inventory, recipe suggestions, and app settings.

Pantry List Screen



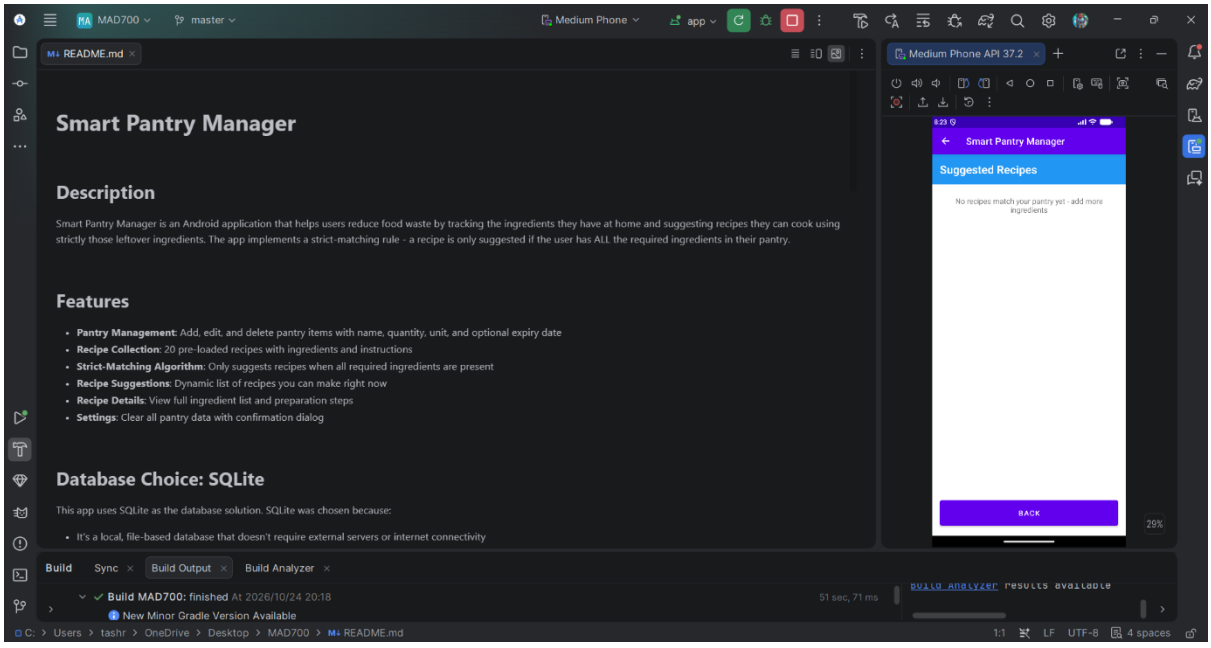
Caption: Your pantry inventory laid out clearly with quantities, units, and expiration tracking, plus an easily accessible delete button for each entry.

Add Ingredient Screen



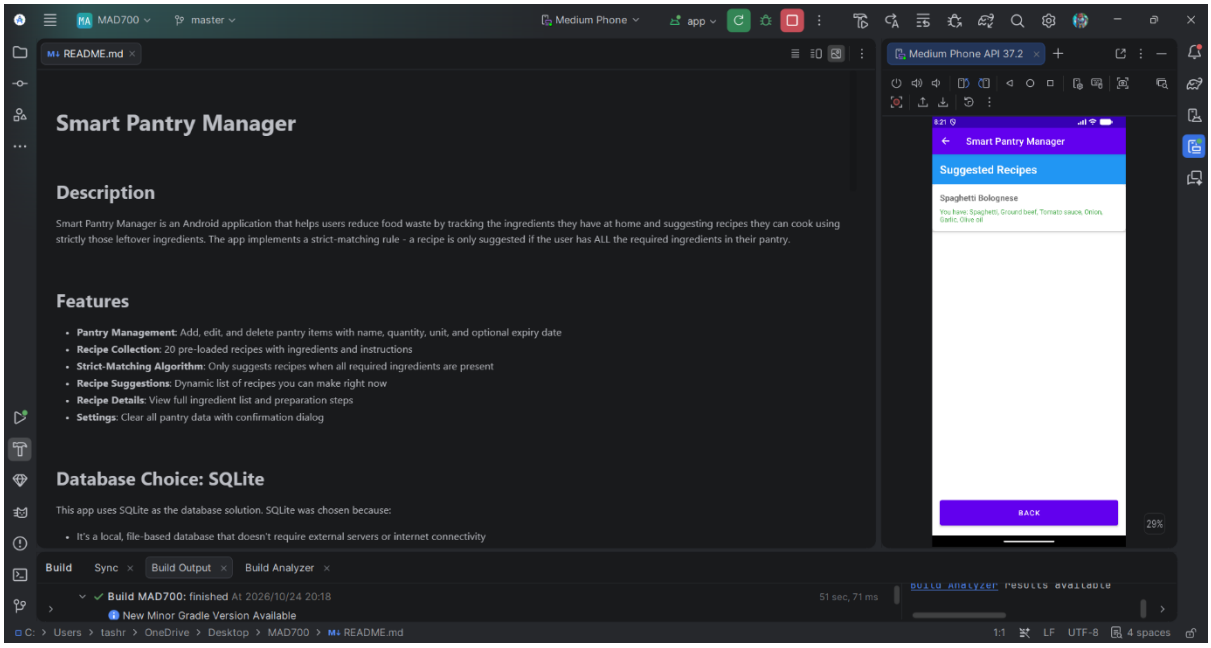
Caption: The form used for adding or updating ingredients, featuring fields for name, quantity, unit, and expiration date alongside save and cancel options.

Suggested Recipes - No Matches



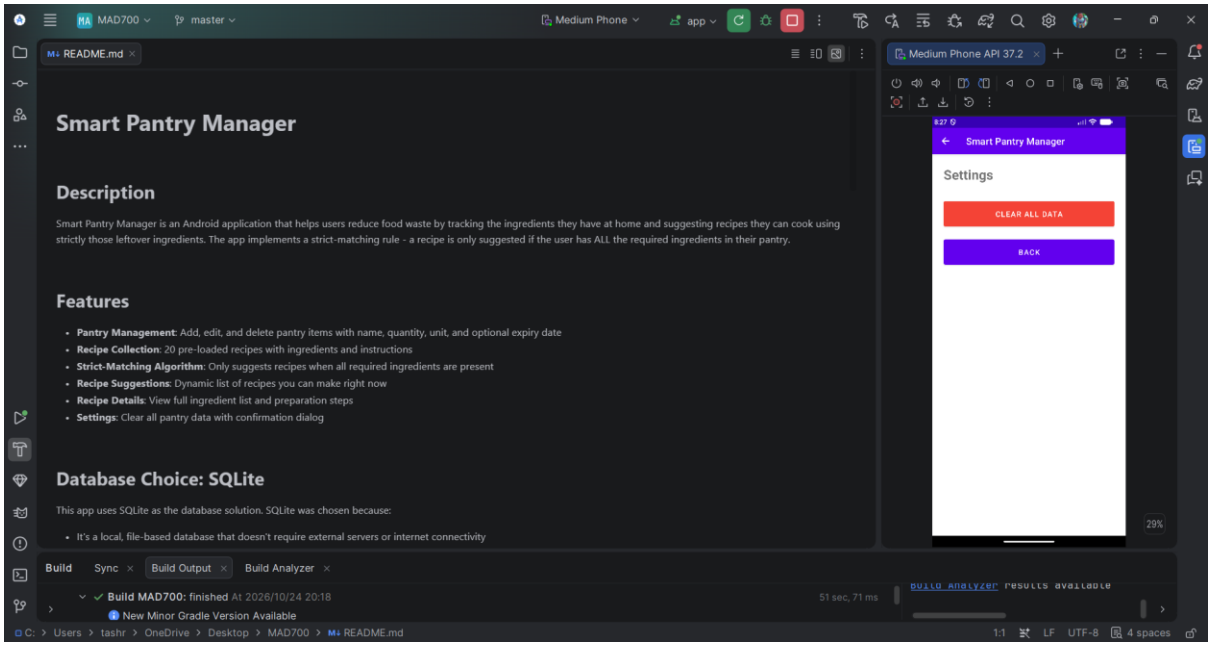
Caption: Friendly feedback showing your current pantry items when no recipes quite match what you have available.

Suggested Recipes - With Matches



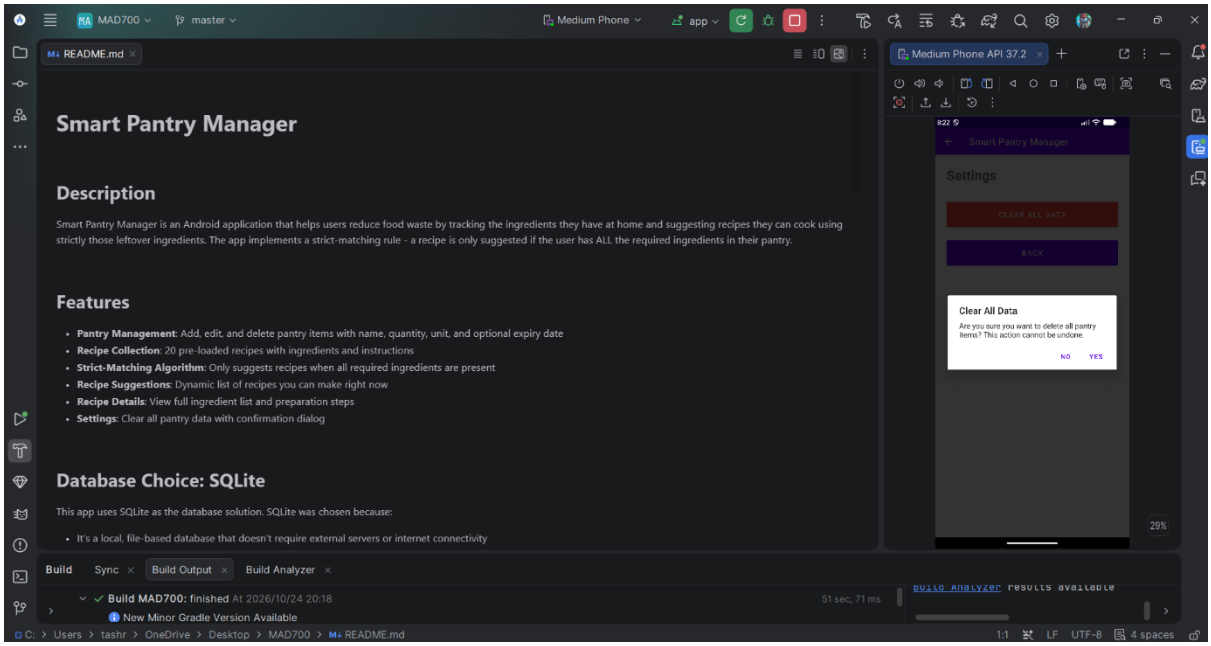
Caption: A targeted list of recipes you can whip up right away using only the ingredients currently in your pantry.

Settings Screen



Caption: The settings view featuring a clear data option to wipe your pantry clean if you want a fresh start.

Delete Confirmation Dialog



Caption: A protective confirmation prompt that pops up before clearing your entire pantry to prevent any accidental data loss.

Key Code Snippets

Snippet 1: Strict-Matching Algorithm (RecipeMatcher.java)

Java

```
public List<Recipe> findMatchingRecipes(List<PantryItem> pantryItems) {
    List<Recipe> matchingRecipes = new ArrayList<>();

    // Create a list of pantry item names for easy matching
    List<String> pantryNames = new ArrayList<>();
    for (PantryItem item : pantryItems) {
        pantryNames.add(item.getName().toLowerCase().trim());
    }

    for (Recipe recipe : allRecipes) {
        String[] recipeIngredients = recipe.getIngredients().split(", ");
        List<String> matching = new ArrayList<>();
        List<String> missing = new ArrayList<>();
```

```
        for (String ingredient : recipeIngredients) {
            String ingredientLower = ingredient.toLowerCase().trim();
            boolean found = false;
            for (String pantryName : pantryNames) {
                // Flexible matching: check if pantry name contains ingredient or vice versa
                // Also handle plural forms by removing trailing 's'
                String pantryRoot = pantryName.endsWith("s") ? pantryName.substring(0,
pantryName.length() - 1) : pantryName;
                String ingredientRoot = ingredientLower.endsWith("s") ?
ingredientLower.substring(0, ingredientLower.length() - 1) : ingredientLower;

                if (pantryName.contains(ingredientLower) ||
                    ingredientLower.contains(pantryName) ||
                    pantryRoot.contains(ingredientRoot) ||
                    ingredientRoot.contains(pantryRoot)) {
                    matching.add(ingredient);
                    found = true;
                    break;
                }
            }
            if (!found) {
                missing.add(ingredient);
            }
        }

        // Strict-matching: only add recipes that have ALL ingredients present
        if (missing.isEmpty()) {
            Recipe matchedRecipe = new Recipe(
                    recipe.getName(),
                    recipe.getIngredients(),
```

```
                recipe.getInstructions(),
                String.join(", ", matching),
                String.join(", ", missing)
            );
            matchingRecipes.add(matchedRecipe);
        }
    }

    return matchingRecipes;
}
```

Explanation: This method is the beating heart of the app's logic. It enforces a strict-matching policy by verifying that every single ingredient a recipe calls for is present in your pantry. To keep things user-friendly, the algorithm is smart enough to handle capitalization, extra spaces, and common plurals (like matching "egg" to "eggs"). If a recipe has zero missing ingredients, it makes the cut.

Snippet 2: Database Helper CRUD Operations (DatabaseHelper.java)

Java

```
public long addPantryItem(PantryItem item) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(COLUMN_NAME, item.getName());
    values.put(COLUMN_QUANTITY, item.getQuantity());
    values.put(COLUMN_UNIT, item.getUnit());
    values.put(COLUMN_EXPIRY, item.getExpiryDate());
    long id = db.insert(TABLE_PANTRY, null, values);
    db.close();
    return id;
}

public List<PantryItem> getAllPantryItems() {
```

```
    List<PantryItem> items = new ArrayList<>();
    SQLiteDatabase db = this.getReadableDatabase();
    Cursor cursor = db.query(TABLE_PANTRY, null, null, null, null, null, COLUMN_NAME
+ " ASC");

    if (cursor.moveToFirst()) {
        do {
            PantryItem item = new PantryItem();
            item.setId(cursor.getLong(cursor.getColumnIndexOrThrow(COLUMN_ID)));

item.setName(cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_NAME)));

item.setQuantity(cursor.getInt(cursor.getColumnIndexOrThrow(COLUMN_QUANTITY)));
            item.setUnit(cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_UNIT)));

item.setExpiryDate(cursor.getString(cursor.getColumnIndexOrThrow(COLUMN_EXPIRY))
);
            items.add(item);
        } while (cursor.moveToNext());
    }
    cursor.close();
    db.close();
    return items;
}

public int updatePantryItem(PantryItem item) {
    SQLiteDatabase db = this.getWritableDatabase();
    ContentValues values = new ContentValues();
    values.put(COLUMN_NAME, item.getName());
    values.put(COLUMN_QUANTITY, item.getQuantity());
    values.put(COLUMN_UNIT, item.getUnit());
```

```
    values.put(COLUMN_EXPIRY, item.getExpiryDate());
    int rowsAffected = db.update(TABLE_PANTRY, values, COLUMN_ID + " = ?", new
String[]{String.valueOf(item.getId())});
    db.close();
    return rowsAffected;
}

public int deletePantryItem(long id) {
    SQLiteDatabase db = this.getWritableDatabase();
    int rowsDeleted = db.delete(TABLE_PANTRY, COLUMN_ID + " = ?", new
String[]{String.valueOf(id)});
    db.close();
    return rowsDeleted;
}
```

Explanation: These functions power the app's local data persistence, handling all Create, Read, Update, and Delete actions using SQLite. Extending SQLiteOpenHelper, the class manages database setup and versions safely. Each method opens the database connection, executes the required command, and closes it right away to avoid resource leaks.

Snippet 3: Custom Adapter with Delete Button (PantryAdapter.java)

Java

```
public class PantryAdapter extends ArrayAdapter<PantryItem> {

    private OnDeleteClickListener deleteListener;

    public interface OnDeleteClickListener {
        void onDeleteClick(PantryItem item);
    }

    public void setOnDeleteClickListener(OnDeleteClickListener listener) {
        this.deleteListener = listener;
    }
```

```
    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        PantryItem item = getItem(position);

        if (convertView == null) {
            convertView = LayoutInflater.from(getContext()).inflate(R.layout.item_pantry,
parent, false);
        }

        TextView tvName = convertView.findViewById(R.id.tvItemName);
        TextView tvQuantity = convertView.findViewById(R.id.tvItemQuantity);
        TextView tvExpiry = convertView.findViewById(R.id.tvItemExpiry);
        Button btnDelete = convertView.findViewById(R.id.btnDelete);

        tvName.setText(item.getName());
        tvQuantity.setText(item.getQuantity() + " " + (item.getUnit() != null ? item.getUnit() :
""));
        tvExpiry.setText("Expires: " + item.getExpiryDate());

        btnDelete.setOnClickListener(v -> {
            if (deleteListener != null) {
                deleteListener.onDeleteClick(item);
            }
        });

        return convertView;
    }
}
```

Explanation: This custom ArrayAdapter bridges the pantry data with the ListView layout. By utilizing an interface callback for the delete button, it allows the parent Activity to smoothly catch deletion events. It also takes advantage of view recycling (convertView) to keep scrolling performance smooth and snappy.

Challenges and Solutions

Challenge 1: Gradle Configuration and JDK Compatibility

Problem: Right at the start, the project refused to build due to version mismatches. Gradle 8.2 clashed with JDK 21, and AndroidX dependencies weren't properly enabled.

Solution:

- Bumped the Android Gradle Plugin up from 8.1.0 to 8.3.0
- Rolled back the Gradle wrapper from 9.7.1 to 8.5 to ensure stable compatibility
- Upgraded Java compatibility settings in build.gradle from version 8 to 17
- Added a gradlew.properties file containing android.useAndroidX=true
- These adjustments cleared up the build errors and got the project compiling smoothly.

Challenge 2: File Locking During Build

Problem: The build process kept throwing frustrating "Unable to delete directory" errors because previous build tasks or active Gradle daemons had locked onto project files.

Solution:

- Killed hanging Gradle daemons using gradlew --stop
- Manually wiped out the stubborn build directories (.gradle, app/build)
- Invalidated caches within Android Studio and performed a restart
- This cleared the file locks and allowed clean builds to run without interference.

Challenge 3: Recipe Matching Not Working

Problem: At first, no recipes were showing up even when all required components were sitting in the pantry. The logic was too rigid, demanding exact text matches that failed over simple things like plurals.

Solution:

- Upgraded the matching routine to handle case-insensitivity and trim excess whitespace
- Added basic plural handling by stripping trailing "s" characters during comparisons
- Introduced substring checks so partial word matches work seamlessly (like catching "egg" inside "eggs")

- Included helpful debug views to show which items were successfully recognized
- This made the matching engine forgiving without breaking the core strict-matching rule.

Challenge 4: Delete Functionality UX

Problem: The initial design relied on a long-press gesture to delete pantry items—something users would never guess how to do naturally.

Solution:

- Added a clean, clearly labeled red delete button directly onto each item row
- Implemented a clean interface pattern inside the adapter to catch clicks on those buttons
- Restructured the list item layout horizontally to accommodate the button cleanly
- This transformed a hidden gesture into an obvious, frictionless action.

Conclusion and Reflection

What Was Learned

1. **Android Development Basics**: Got hands-on practice with Activities, Intents, adapters, and managing the core app lifecycle
2. **SQLite Integration**: Learned how to implement local storage reliably using helper classes and standard CRUD methods
3. **UI/UX Crafting**: Focused on building clean layouts with CardViews and intuitive navigation paths
4. **Build Tooling**: Gained a deeper understanding of Gradle configurations, dependencies, and project architecture
5. **Algorithmic Thinking**: Designed a pragmatic matching algorithm that balances strict rules with real-world usability
6. **Version Control**: Used Git consistently to track feature additions and manage project history safely

What Worked Well

- The strict-matching engine does a fantastic job of surfacing genuinely cookable recipes
- SQLite delivers reliable, fast local persistence without needing a remote server
- Keeping responsibilities cleanly separated across different Activities keeps the codebase organized and easy to maintain
- Custom adapters handle dynamic lists efficiently

- Thorough input validation keeps garbage data out of the database

What Could Be Improved

1. **Dynamic Recipes**: Right now, recipes are hardcoded in Java. Moving them into the database would let users add their own custom meals
2. **Quantity Awareness**: The matching engine looks for ingredients by name, but doesn't check if you actually have enough of them in stock
3. **Expiration Alerts**: The app records expiration dates, but doesn't send out proactive warnings when food is about to spoil
4. **Search and Filter**: Adding a quick search bar would make navigating large pantries much faster
5. **Visual Polish**: Including recipe photos would make the suggestions feel a lot more inviting
6. **Data Backup**: Adding local backups or cloud syncing would protect users from losing data if they change devices

Future Enhancements

Given more time to develop the app further, I would:

- Build a full recipe management dashboard where users can create, edit, and share custom recipes
- Upgrade the matching algorithm to factor in exact ingredient quantities
- Implement push notifications for expiring groceries
- Add robust search, filter, and sorting tools
- Hook up a lightweight cloud backend for cross-device synchronization
- Introduce nutritional summaries and an automated shopping list creator for missing ingredients

References

Android Documentation

- Android Developers. (2024). *SQLite and SQLiteOpenHelper*. https://developer.android.com/training/data-storage/sqlite
- Android Developers. (2024). *RecyclerView*. https://developer.android.com/guide/topics/ui/layout/recyclerview
- Android Developers. (2024). *Activities*. https://developer.android.com/guide/components/activities/intro-activities

Tutorials and Resources

- Vogella, L. (2024). *Android Development Tutorial*. http://www.vogella.com/tutorials/Android/article.html
- Androidhive. (2024). *Android SQLite Database Tutorial*. https://www.androidhive.info/2011/11/android-sqlite-database-tutorial/

Tools and Libraries

- Android Studio. (2024). *Official IDE for Android Development*. https://developer.android.com/studio
- Gradle. (2024). *Build Automation Tool*. https://gradle.org/